

High Level Architecture

High-Level Architecture · v2.0

What This Is

A composition framework for building AI-powered voice and chat systems across any domain — government services, enterprise, startups, or public institutions. Instead of building AI from scratch, deployers assemble a system from **7 DPG building blocks** and configure it for their domain using a **Domain Configuration Kit**.

The framework separates what changes every deployment (domain knowledge, connectors, conversation design, safety policy) from what stays the same (the 7 DPGs and their runtime behavior).

Architecture at a Glance

Ref: <https://ai-diffusion-dpg.lovable.app/>



The 7 DPG Building Blocks

1. Knowledge Engine

Role: Assembles the complete prompt for each LLM call.

Pulls conversation history from the Memory Layer, retrieves relevant knowledge chunks via RAG, applies domain vocabulary and language normalization, and constructs the system prompt with persona and guardrail instructions. Everything that shapes *what the LLM sees* lives here.

Functional blocks inside:

Block	What It Does
NLU Processor	Intent classification, entity extraction, sentiment analysis
Glossary & Domain Vocabulary	Maps colloquial terms to canonical domain concepts
Language Normalization Layer	Dialect detection, code-switching (Hinglish), transliteration
Static Knowledge Base	Semantic retrieval over ingested domain documents (RAG)
Multimodal Input Handler	Processes images, PDFs, audio clips mid-conversation

2. Memory Layer

Role: Manages all state across three scopes — the system remembers who the user is, what was said, and where they are in their journey.

Scope	Lifespan	Contains	Example
Turn	Current processing cycle	Raw input, intermediate results	Parse results, tool call outputs
Session	Single conversation	Turn history, confirmed entities, workflow step	"User asked about electrician roles in Hubli"
Persistent	Indefinite (until user deletes)	User profile, journey history, outcomes	Skills, location, past interactions across calls

Also provides a pub/sub feature for broadcast scenarios (mass outage updates, disaster alerts pushed to all active sessions).

Functional blocks inside:

Block	What It Does
Session Memory Handler	Conversational state within a single call — history, entities, unresolved intents
User Profile Store	Cross-session persistent memory — preferences, past interactions, language
Task State Manager	Multi-step workflow progress — knows what step is completed, what is pending
Broadcast / Incident State	System-wide shared state — pushes known issues across all active sessions

3. Trust Layer

Role: Enforces all policy, safety, compliance, and escalation rules. Mandatory — every input and every output passes through this layer.

Operates as five configurable rule categories within a single evaluation engine:

Category	What It Does	Example
Content rules	Blocks harmful inputs, enforces topic boundaries	PII detection, abuse filtering
Output rules	Verifies LLM response before delivery	Blocks "guaranteed placement", enforces source citation
Consent rules	Manages user consent flows, DPDP compliance	Consent required before storing profile or submitting form
Escalation rules	Detects when AI must hand off to a human	Low confidence, user distress, evaluation loop ≥ 3
Topic firewall	Topics that always go to humans regardless	Legal disputes, harassment complaints

Functional blocks inside:

Block	What It Does
-------	--------------

Guardrails Engine	Pre/post-processing — blocks harmful inputs, enforces topic boundaries, detects jailbreaks
Consent & Compliance Handler	Manages user consent flows, DPDP Act compliance, logs consent events
HITL Handler	Confidence-based human escalation — detects when AI should hand off
Priority & Escalation Classifier	Real-time urgency detection — medical emergency, vulnerable user — overrides routing
Topic Firewall	Business rule enforcement — certain topics always routed to humans

4. Agent Core (*Orchestrator*)

Role: The brain of the framework. Orchestrates all other DPGs and executes the LLM call.

The Agent Core is both the intelligence layer (it calls the LLM) and the orchestrator (it coordinates Knowledge Engine, Trust Layer, Action Gateway, and Memory Layer in the correct sequence for every turn). There is no separate orchestration container — the Agent Core *is* the runtime process.

What it does on every turn:

None

Input arrives

- Reads state (Memory Layer)
- Checks safety (Trust Layer – input pass)
- Assembles prompt (Knowledge Engine)
- Calls LLM → receives response + tool-call decisions
- If tools needed → invokes Action Gateway → feeds results back to LLM

LLM

- Checks safety (Trust Layer – output pass)
- Delivers response via channel
- Updates state (Memory Layer – async)
- Emits events (Learning Layer – async)

Routing logic: Instead of a separate routing agent deciding which block to call, routing happens inside the LLM call via tool definitions. The LLM decides whether to query an external API, retrieve more knowledge, or respond directly — based on the domain configuration. Hard routing rules (e.g., "harassment complaints always go to humans") are handled by the Trust Layer *before* the LLM is ever called.

Functional blocks inside:

Block	What It Does
LLM Inferencing Wrapper	Standard interface over any LLM — prompt construction, token management, retry, fallback models
Manager Agent	Rules-first routing with LLM fallback — routes to right blocks per intent
Orchestration Config Layer	YAML → runtime wiring — the entire agent is defined in one config file

5. Action Gateway

Role: Executes all interactions with systems outside the framework. The LLM never touches external systems directly — it expresses intent through tool calls, and this layer executes them.

Every external integration follows the same pattern:

None

```
LLM says: "Call check_eligibility with {state: UP, income: <2.5L}"
  → Action Gateway executes the API call
  → Returns normalized result to Agent Core
  → Agent Core feeds result back to LLM for response generation
```

Three connector categories, one unified pattern:

Category	Purpose	Examples
Read connectors	Query external systems	Check eligibility, fetch mandi prices, poll job listings

Write connectors	Take actions in external systems	Submit application, register for training, file complaint
Identity connectors	Verify user identity mid-conversation	Aadhaar OTP, service number check, policy ID verification

Write connectors and identity connectors are always gated by the Trust Layer's consent rules — the system must confirm with the user before executing.

Functional blocks inside:

Block	What It Does
Dynamic Data Connector	Live API / MCP calls, normalized output at query time
Actions Executor	Intent → real-world action — form submission, DB writes, appointment booking
Identity & Auth Handler	Mid-conversation identity verification — Aadhaar OTP, service number check

6. Reach Layer

Role: Manages how the system reaches users (inbound and outbound) and how conversations move across channels without losing context.

Mode	What It Does	Driven By
Inbound	Receives calls/messages, normalizes input, passes to Agent Core	User-initiated
Outbound	Runs campaigns — re-engagement callbacks, alert broadcasts, follow-up check-ins	Scheduled by domain config
Channel switch	Moves conversation from voice to WhatsApp (or back) without losing context	Agent Core triggers based on user signals

Channel continuity rule: The user's phone number is the universal key. When a conversation moves across channels, the Memory Layer provides full context. The user never restarts.

Functional blocks inside:

Block	What It Does
Campaign Orchestrator	Outbound list management, scheduling, retry logic, do-not-disturb windows
Channel Adapter	Normalizes input/output across VOIP, WhatsApp, Web, Mobile SDK
Handoff Manager	Manages cross-channel transitions with full context preservation

7. Learning Layer

Role: Logs everything, measures quality, and feeds the improvement loop. Runs entirely async — never in the response path.

Three output streams:

Stream	What It Captures	What It Feeds
Audit log	Every prompt, response, routing decision, tool call, consent event — with timestamps	Government compliance, legal defensibility
Quality evaluation	Groundedness, relevance, hallucination detection, task completion scoring	Dashboard for deployment team, alerts on degradation
Signal collection	Explicit feedback ("that's wrong"), implicit signals (repeated question, call drop), post-action outcomes	Knowledge base updates, guardrail tuning, connector quality scoring

The improvement loop: Signals don't just sit in dashboards — they feed back into the Domain Configuration Kit. Low groundedness scores flag knowledge gaps. High escalation rates flag routing problems. Post-action mismatches flag data source quality issues. This loop is where production systems succeed or fail.

Functional blocks inside:

Block	What It Does
-------	--------------

LLM Observability	Full call logging — every prompt, response, routing decision, latency, cost, model version
Evals Framework	Automated response quality scoring — groundedness, relevance, hallucination detection
Feedback Signal Collector	Captures explicit and implicit feedback signals, structures as training data
Outcome Tracker	Structured outcome capture after each call — resolved, pending, action taken

The Domain Configuration Kit

The 7 DPGs are reusable across every deployment. The Domain Configuration Kit is what makes each deployment *specific* — and it is where 80% of the effort lives.

A deployer must configure five areas. These are not optional.

Area 1: Knowledge Corpus

The static and semi-static information the system needs to answer questions accurately. Includes domain documents (scheme guidelines, product catalogs, clinical protocols), a domain glossary mapping colloquial terms to canonical concepts, and language/dialect configuration.

What makes it work: Covers 90%+ of expected queries. Is current. Is chunked for retrieval. Includes negative knowledge ("this scheme was discontinued").

What goes wrong: Raw policy documents dumped without curation. No FAQs from real user questions. Stale information. No retrieval quality testing before launch.

Area 2: Connectors & Data Sources

The live APIs and external systems the agent needs to interact with. Each connector is declared with its endpoint, authentication, input parameters, timeout, retry policy, consent requirements, and fallback response.

What makes it work: Every connector has a tested fallback. Input parameters are mapped to either user profile fields or conversation-extracted entities. Write connectors always require consent.

What goes wrong: Government APIs with no sandbox environment. Unstable uptime. No fallback configured — agent hangs when API is down.

Area 3: Conversation Design

The persona, behavior rules, mental model of the user, and multi-step workflows. This is where domain expertise matters most — understanding *how users actually talk about their problems* and designing the system's responses accordingly.

What makes it work: Built from real user conversations, not assumed ones. Tested with target users before launch. Includes explicit instructions for edge cases.

What goes wrong: Persona designed by engineers, not domain experts. No user mental model. Workflows that assume linear progression when real conversations are messy.

Area 4: Trust & Policy

Domain-specific safety rules, compliance requirements, escalation triggers, and topic firewalls. Configures the Trust Layer for the specific use case.

What makes it work: Escalation rules calibrated to actual risk levels. Always-human topics identified from domain knowledge. Consent flows that feel natural in conversation, not bureaucratic.

What goes wrong: Over-restrictive guardrails that block legitimate queries. Under-specified escalation rules that send too many or too few conversations to humans.

Area 5: Success Criteria & Evaluation Rules

What "working" means for this deployment, measured automatically. Defines primary metrics, quality checks, drop-off taxonomy, and re-engagement triggers.

What makes it work: Primary metric reflects actual user value (e.g., "informed decision rate"), not vanity metrics (e.g., "calls handled"). Quality checks run on every turn, not sampled.

What goes wrong: Measuring conversion rate instead of user outcome. No drop-off taxonomy — can't distinguish "user got what they needed and left" from "user gave up in frustration."

Runtime Considerations

Since the Agent Core is both the orchestrator and the LLM execution layer, runtime behavior is a property of how it operates — not a separate component.

Latency Budget (Voice-First)

Total target: 800–1200ms per turn (excluding upstream ASR and downstream TTS).

Phase	Budget	What Happens	DPGs Active
Input processing	≤100ms	State read → Safety check → Prompt assembly	Memory Layer · Trust Layer · Knowledge Engine
Execution	≤700ms	LLM call(s) + tool execution if needed	Agent Core · Action Gateway
Output processing	≤50ms	Safety check → Channel delivery	Trust Layer · Reach Layer
Post-turn	0ms (async)	State write → Observability emit	Memory Layer · Learning Layer

Most turns require one LLM call. Turns involving external tool calls require two (LLM → tool → LLM). The Agent Core manages this within the budget, falling back to a faster model if the primary model is slow.

Scaling Model

The Agent Core is stateless — all state lives in the Memory Layer's backing stores. Instances scale horizontally based on concurrent sessions. Each instance handles independent sessions with no cross-instance coordination needed.

Deployment Flexibility

The framework is infrastructure-agnostic. The Agent Core runs as a container, serverless function, or long-running process on any cloud, on-prem, or edge environment. The only hard dependencies are the Memory Layer's backing stores (a fast key-value store for session state, a database for persistent state) and network access to the configured LLM provider and external APIs.

What This Architecture Does Not Cover

Intentionally out of scope for the composition framework, but critical for production:

- **ASR/TTS pipeline** — the framework receives text and returns text
 - **Model training & fine-tuning** — uses foundation models via API
 - **Infrastructure provisioning** — infrastructure-agnostic by design
 - **Multi-tenancy & cost attribution** — platform engineering concern
 - **Testing & simulation** — needed before every launch, not yet formalized
 - **Versioning & rollback** — config changes need version control and rollback capability
-

From 25 Blocks to 7 DPGs — The Mapping

Original 25-Block Architecture	Consolidated As
NLU Processor, Glossary & Domain Vocab, Language Normalization, Static Knowledge Base, Multimodal Input Handler	Knowledge Engine
Session Memory Handler, User Profile Store, Task State Manager, Broadcast/Incident State	Memory Layer
Guardrails Engine, Consent & Compliance Handler, HITL Handler, Priority & Escalation Classifier, Topic Firewall	Trust Layer
LLM Inferencing Wrapper, Manager Agent, Orchestration Config Layer	Agent Core
Dynamic Data Connector, Actions Executor, Identity & Auth Handler	Action Gateway
Campaign Orchestrator	Reach Layer

LLM Observability, Evals Framework, Feedback Signal Collector, Conversation Outcome Tracker	Learning Layer
--	-----------------------

Same capabilities. 7 DPGs instead of 25 blocks. Clearer boundaries. One orchestrator, not a distributed puzzle.

Core Design Principles

Core Design Principles

From 25 Blocks to 7 DPGs — Why and How

Part 1: Why 25 Blocks Doesn't Work

The original architecture defined 25 independent building blocks across 6 tiers. While comprehensive in coverage, it suffered from five structural problems that would prevent real-world adoption.

Problem 1: False Equivalence

The 25 blocks treated fundamentally different things as if they were the same kind of component. A Session Memory Handler (a standard engineering pattern any developer would implement) was given the same architectural weight as a Consent & Compliance Handler (a genuine public good implementing DPDP Act requirements). Calling everything a "DPG" diluted the meaning of the term and made it harder for adopters to understand what was truly novel versus what was standard engineering.

Problem 2: Sequential Latency Explosion

The original architecture implied that each block was a discrete runtime service, invoked sequentially. For a single user turn, the flow would traverse NLU Processor → Language Normalization → Glossary Mapping → Guardrails (input) → Manager Agent → LLM Wrapper → Guardrails (output) — at minimum 7 sequential hops before counting knowledge retrieval, tool calls, or state management. At 100–150ms per hop, the total latency would reach 700ms–1s *before* the LLM call, blowing the entire voice-first latency budget of 800–1200ms.

Problem 3: Prompt-Time Concerns Modeled as Services

NLU processing, glossary normalization, language normalization, and input guardrail checking are not independent services that need their own network endpoints. They are *concerns that shape how the LLM prompt is constructed*. A single well-constructed LLM call with the right system prompt can handle intent classification, entity extraction, domain vocabulary mapping, sentiment detection, and response generation — all at once. Modeling these as separate services creates unnecessary complexity, latency, and failure modes.

Problem 4: Unclear Orchestration

The architecture mentioned a Manager Agent and an Orchestration Config Layer, but never defined the actual runtime process that receives a user message and coordinates everything. The YAML configuration was presented as if it *were* the orchestrator, but YAML is configuration, not execution. The hardest design decision — what is the runtime container, how does it sequence calls, how does it handle failures — was deferred to the implementer.

Problem 5: Adoption Complexity

A government IT vendor evaluating 25 blocks faces a cognitive overload problem. Which blocks are mandatory? Which are optional? How do they connect? What's the minimum viable deployment? The original architecture answered these questions (6 universal blocks + 19 selective), but the sheer number of components made the system feel heavyweight and intimidating — the opposite of the "oversimplify use-case adoption" goal stated in the vision.

Part 2: The Merge Logic — Block by Block

Each merge follows one of four principles:

Principle	What It Means	Example
Prompt collapse	Multiple blocks that are actually prompt-time concerns, not runtime services	NLU + Glossary + Language Norm → all shape the prompt
Scope unification	Multiple blocks that manage the same thing at different scopes	Session Memory + User Profile + Task State → all manage state
Policy consolidation	Multiple blocks that enforce different types of the same kind of rule	Guardrails + Consent + HITL + Escalation + Firewall → all enforce policy
Pattern deduplication	Multiple blocks that follow the same technical pattern with different targets	Dynamic Data Connector + Actions Executor + Identity Handler → all call external APIs

Merge 1: Knowledge Engine

Principle: Prompt collapse

Original Block	Why It Merges
NLU Processor	Intent classification and entity extraction happen inside the LLM call when the prompt is well-constructed — not as a separate pre-processing service
Glossary & Domain Vocabulary	Term normalization is a prompt-time text transformation applied before the LLM sees the input
Language Normalization Layer	Dialect detection, code-switching, and transliteration are pre-processing steps during prompt assembly
Static Knowledge Base	RAG retrieval provides context chunks that are injected into the prompt
Multimodal Input Handler	Image/PDF processing extracts text context that gets added to the prompt

The insight: All five blocks answer one question — *"What should the LLM see when it processes this turn?"* They are five steps in prompt assembly, not five independent services. Making them separate services adds 5 network hops and 5 failure modes for zero architectural benefit.

What stays distinct inside the Knowledge Engine: RAG retrieval still involves a vector database query (one fast network call, <50ms). Language normalization may wrap an external provider (Bhashini). But these are implementation details within one functional unit, not separate architectural blocks.

Merge 2: Memory Layer

Principle: Scope unification

Original Block	Why It Merges
Session Memory Handler	Manages state at session scope (current conversation)

User Profile Store	Manages state at persistent scope (cross-session user identity)
Task State Manager	Manages state at journey scope (multi-step workflow progress)
Broadcast / Incident State	Manages state at system scope (push updates to all sessions)

The insight: These four blocks all manage state — they differ only in scope (turn / session / persistent / broadcast) and lifespan. The read/write patterns are identical: read on turn start, write on turn end. The backing stores may differ (in-memory for turn, Redis for session, Postgres for persistent), but that is a storage configuration choice, not an architectural boundary.

Why a single block with multiple scopes is better: State management has cross-scope concerns. When a user confirms a new piece of information in a session (session scope), it should update their persistent profile (persistent scope) and may affect their workflow progress (journey scope). If these are separate blocks, coordinating cross-scope updates requires inter-block communication — adding complexity. A single Memory Layer with three scopes handles this internally.

Merge 3: Trust Layer

Principle: Policy consolidation

Original Block	Why It Merges
Guardrails Engine	Enforces content safety rules — topic boundaries, PII detection, abuse filtering
Consent & Compliance Handler	Manages consent flows and DPDP Act compliance
HITL Handler	Detects when AI should hand off to a human based on confidence
Priority & Escalation Classifier	Detects urgency — medical emergency, vulnerable user — overrides routing
Topic Firewall	Routes certain topics to humans regardless of AI confidence

The insight: All five blocks answer one question — *"Is it safe and appropriate for the AI to handle this?"* They differ in *what kind* of policy they enforce (content safety vs. consent vs. escalation vs. topic restriction), but the runtime pattern is identical: evaluate input or output against a set of rules, return pass/block/escalate.

Why consolidation matters here specifically: Safety cannot have gaps. If guardrails, escalation, and topic firewall are separate services, a failure in one (network timeout, misconfiguration) could let unsafe content through while the others still pass. A single Trust Layer with five rule categories evaluates all policies in one pass — if any category blocks, the entire input/output is blocked. This is a security architecture principle: the safety boundary must be a single, non-bypassable gate.

What stays distinct inside the Trust Layer: The rule categories remain separately configurable. A deployer configures content rules, consent rules, escalation rules, and topic firewall rules independently. But at runtime, they execute as one evaluation pass.

Merge 4: Agent Core

Principle: The orchestrator and the LLM caller are the same thing

Original Block	Why It Merges
LLM Inferencing Wrapper	Makes the actual LLM API call, manages tokens, retry, fallback
Manager Agent	Routes queries to the right blocks based on intent
Orchestration Config Layer	Reads YAML configuration and wires the runtime

The insight: The original architecture separated "deciding what to do" (Manager Agent), "doing the LLM call" (Inferencing Wrapper), and "reading the configuration" (Config Layer) into three blocks. But in practice, the LLM call *is* the routing decision — modern LLMs with tool-use capabilities decide whether to call an API, retrieve knowledge, or respond directly based on the system prompt. The Manager Agent's routing logic is expressed as tool definitions in the LLM call, not as a separate pre-routing step.

Why the Agent Core is the orchestrator: There is no useful distinction between "the thing that calls the LLM" and "the thing that coordinates all other blocks." The Agent Core reads the configuration, assembles the prompt (via Knowledge Engine), checks safety (via Trust Layer), makes the LLM call, executes tool calls (via Action Gateway), updates state (via Memory Layer), and

emits signals (via Learning Layer). Separating orchestration from LLM execution creates an empty abstraction — a block whose only job is to call another block.

Merge 5: Action Gateway

Principle: Pattern deduplication

Original Block	Why It Merges
Dynamic Data Connector	Calls external APIs to read data (eligibility checks, market prices)
Actions Executor	Calls external APIs to write data (form submission, application filing)
Identity & Auth Handler	Calls external APIs to verify identity (Aadhaar OTP, DigiLocker)

The insight: These three blocks follow the exact same technical pattern: receive intent + parameters from the LLM → call an external API with authentication → handle timeout/retry/fallback → return normalized result. They differ in *what* they call (read API vs. write API vs. identity API) and *what policy gates apply* (write connectors require consent, identity connectors require verification), but these differences are configuration, not architecture.

The connector model: Instead of three block types, the Action Gateway has one connector pattern with three categories (read / write / identity). Each connector is declared in the YAML with its endpoint, authentication, parameters, timeout, consent requirements, and fallback response. Adding a new external integration means adding a connector configuration — not building a new block.

Merge 6: Reach Layer

Principle: Channel management is one concern

Original Block	Why It Merges
Campaign Orchestrator	Manages outbound call campaigns, scheduling, retry logic

(implicit) Channel Adapter	Normalizes input/output across VOIP, WhatsApp, Web
(implicit) Handoff Manager	Manages cross-channel transitions with context preservation

The insight: The original architecture had Campaign Orchestrator as an explicit block but left channel management and cross-channel handoff implicit. In practice, these are all part of one concern: *how does the system reach users and maintain continuity across touchpoints?* Inbound channel normalization, outbound campaign orchestration, and cross-channel handoff all operate on the same user identity (phone number) and the same state (from Memory Layer).

Merge 7: Learning Layer

Principle: Post-hoc analysis is one concern

Original Block	Why It Merges
LLM Observability	Logs every prompt, response, routing decision with timestamps
Evals Framework	Automated quality scoring — groundedness, relevance, hallucination
Feedback Signal Collector	Captures explicit and implicit user feedback
Conversation Outcome Tracker	Records structured post-call outcomes

The insight: All four blocks run async (never in the response path), all process the same raw material (completed turns and sessions), and all feed the same improvement loop. Separating them creates four async pipelines that need to be independently deployed, monitored, and maintained — for no user-facing benefit. A single Learning Layer with three output streams (audit log, quality evaluation, signal collection) is simpler to operate and ensures that all observability data flows through one system.

Part 3: Core Design Principles

These principles govern the architecture and should guide every implementation decision.

Principle 1: Concerns, Not Services

Most architectural components are not independent runtime services — they are concerns that get configured into a small number of actual runtime operations. The primary runtime operations in any turn are: one state read, one prompt assembly, one or two LLM calls, zero or more external API calls, one state write, and one observability emit. Everything else is configuration that shapes these operations, not a separate service invocation.

Test: Before making anything a separate service, ask: *"Does this require its own network call at runtime, or is it a configuration concern that shapes an existing call?"* If the latter, it belongs inside an existing DPG as configuration, not as a new block.

Principle 2: Latency-First Design

Voice-first use cases have a hard latency budget of 800–1200ms from text input to text output (excluding ASR/TTS). Every architectural decision is evaluated against this constraint. Sequential block invocations are eliminated wherever possible. The target is 1–2 LLM calls per turn, not a chain of 6+ service invocations.

Budget allocation:

- Input processing (state read + safety check + prompt assembly): $\leq 100\text{ms}$
- Execution (LLM calls + tool calls): $\leq 700\text{ms}$
- Output processing (safety check + delivery): $\leq 50\text{ms}$
- Post-turn (state write + observability): 0ms (async)

Test: Before adding a synchronous step to the turn lifecycle, ask: *"Does this fit within the remaining latency budget? Can it run async instead?"*

Principle 3: The Trust Layer is a Containment Boundary, Not a Sidecar

The Trust Layer is not optional middleware that can be bypassed. It is a containment boundary — the Agent Core operates *inside* the Trust Layer. Every input passes through safety evaluation before the LLM sees it. Every output passes through policy verification before the user sees it.

This is a security architecture principle: safety enforcement must be a single, non-bypassable gate. If safety checks are distributed across multiple optional blocks, a failure in any one creates a gap. The Trust Layer's dual-pass model (input check + output check) ensures that no content enters or leaves the system without policy evaluation.

Test: Can any code path reach the LLM or the user without passing through the Trust Layer? If yes, the architecture has a safety gap.

Principle 4: Composition Over Construction

The framework does not build AI systems — it composes them. The 7 DPGs are the building blocks. The Domain Configuration Kit is the assembly instruction. The YAML is the interface contract. A deployer never writes code to create a new use case — they configure an existing set of capabilities for their domain.

This means the 7 DPGs must be general enough to serve any domain (healthcare, agriculture, finance, education) while the Domain Configuration Kit must be specific enough to capture the nuances of each domain (clinical protocols vs. crop calendars vs. loan eligibility vs. scholarship criteria).

Test: If a new use case requires a new DPG block (not just new configuration), the architecture has a generality gap. The 7 blocks should be sufficient for any use case; only the configuration changes.

Principle 5: Domain Configuration is the Hard Work

The technical surface area of the framework is small (7 DPGs, one runtime process, one YAML schema). The domain configuration surface area is large — intentionally. Curating the right knowledge base, building reliable data connectors, designing conversation flows that work for real users, calibrating safety rules, and defining meaningful success metrics — this is where 80% of deployment effort goes.

The architecture explicitly separates the composition framework (Layer 1, owned by the core engineering team) from the domain configuration (Layer 3, owned by the deploying organisation). This separation ensures that domain expertise drives deployment quality, not engineering effort.

Test: Is the deployment team spending most of their time on framework engineering or on domain configuration? If the former, the framework is not doing its job.

Principle 6: The LLM is the Router

The original architecture had a separate Manager Agent that routed queries to different blocks using rules-first logic with LLM fallback. In the consolidated architecture, routing happens *inside* the LLM call via tool definitions. The LLM decides whether to query an external API, retrieve more knowledge, escalate to a human, or respond directly — based on the system prompt and available tool definitions.

Hard routing rules (e.g., "harassment complaints always go to humans") are handled by the Trust Layer *before* the LLM call, not by a separate routing agent. This eliminates a sequential hop and leverages the LLM's reasoning capability for soft routing decisions.

Test: Is there a routing decision that cannot be expressed either as a Trust Layer rule (hard routing) or as an LLM tool-use decision (soft routing)? If yes, examine whether it is genuinely a routing problem or a configuration gap in one of these two mechanisms.

Principle 7: State Scope, Not State Blocks

State management is one concern with multiple scopes — not multiple concerns with separate implementations. Turn state (ephemeral), session state (single conversation), persistent state (cross-session profile), and broadcast state (system-wide events) all follow the same read/write pattern. They differ in lifespan and backing store, but these are configuration choices within one Memory Layer.

Cross-scope coordination (a session confirmation updating a persistent profile) happens internally within the Memory Layer, not through inter-block communication. This eliminates a class of consistency bugs that arise when state is distributed across independent blocks.

Test: When a user confirms new information, does it update all relevant scopes atomically? If scopes are managed by different blocks, cross-scope consistency requires explicit coordination — which is complexity that belongs inside a single Memory Layer.

Principle 8: Observability is a Feedback Loop, Not Just Logging

The Learning Layer exists not just to log what happened (audit), but to measure how well it worked (evaluation) and to surface what needs to change (signals). The feedback loop — from

Learning Layer signals to Domain Configuration Kit updates — is where production systems improve over time.

Without this loop, the system is static: it deploys with a fixed knowledge base and fixed guardrails, and degrades as the world changes. With this loop, low groundedness scores trigger knowledge base updates, high escalation rates trigger routing adjustments, and post-action mismatches trigger data source quality reviews.

Test: When the system gives a wrong answer, is there a defined path from detection (Learning Layer) to correction (Domain Configuration Kit update)? If not, the feedback loop is broken.

Principle 9: The Configuration Agent is a Tool, Not a DPG

The Configuration Agent (Tier 1) interviews domain experts and generates YAML configuration. It is a powerful enabler of adoption — but it is not part of the runtime architecture. It runs occasionally (initial setup, major changes), not on every user turn. It could itself be built on the same 7-DPG framework, but it is not one of the 7 DPGs.

This distinction matters because it keeps the runtime architecture clean. The 7 DPGs are the complete set of runtime components. Adding the Configuration Agent as an 8th DPG would blur the boundary between "what runs in production" and "what helps you set up production."

Test: Does this component need to be running for the system to serve users? If not, it is a tool, not a DPG.

Principle 10: Infrastructure is a Deployment Decision, Not an Architectural Layer

The framework is infrastructure-agnostic. The Agent Core runs as a container, serverless function, or long-running process on any cloud, on-prem, or edge environment. The Memory Layer's backing stores (Redis, Postgres, DynamoDB) are deployment choices. The LLM provider (Anthropic, OpenAI, Google, local Ollama) is a configuration setting.

Infrastructure decisions (cloud provider, GPU allocation, networking, data residency) are driven by deployment requirements (latency, scale, compliance, cost) — not by the framework architecture. The framework defines *what* runs; infrastructure defines *where and how* it runs.

Test: Can the same YAML configuration be deployed on AWS, GCP, on-prem, or edge without changing any DPG code? If not, there is an infrastructure coupling that should be abstracted.

Summary: The Consolidation at a Glance

#	DPG	Merge Principle	Blocks Absorbed	Core Insight
1	Knowledge Engine	Prompt collapse	5 blocks	All are steps in prompt assembly, not separate services
2	Memory Layer	Scope unification	4 blocks	All manage state — differ only in scope and lifespan
3	Trust Layer	Policy consolidation	5 blocks	All enforce policy — must be one non-bypassable gate
4	Agent Core	Orchestrator = LLM caller	3 blocks	No useful distinction between orchestration and execution
5	Action Gateway	Pattern deduplication	3 blocks	All follow the same API call pattern with different targets
6	Reach Layer	Channel unification	1 block + 2 implicit	Inbound, outbound, and handoff are one concern
7	Learning Layer	Post-hoc unification	4 blocks	All run async on the same raw material for the same purpose

25 blocks → 7 DPGs. Same capabilities. Fewer moving parts. Clearer boundaries. Faster deployments. Lower latency. Stronger safety.

Use-Case Configuration

Use-Case Configuration

How a Use Case Gets Configured on the Composition Framework

The Core Insight

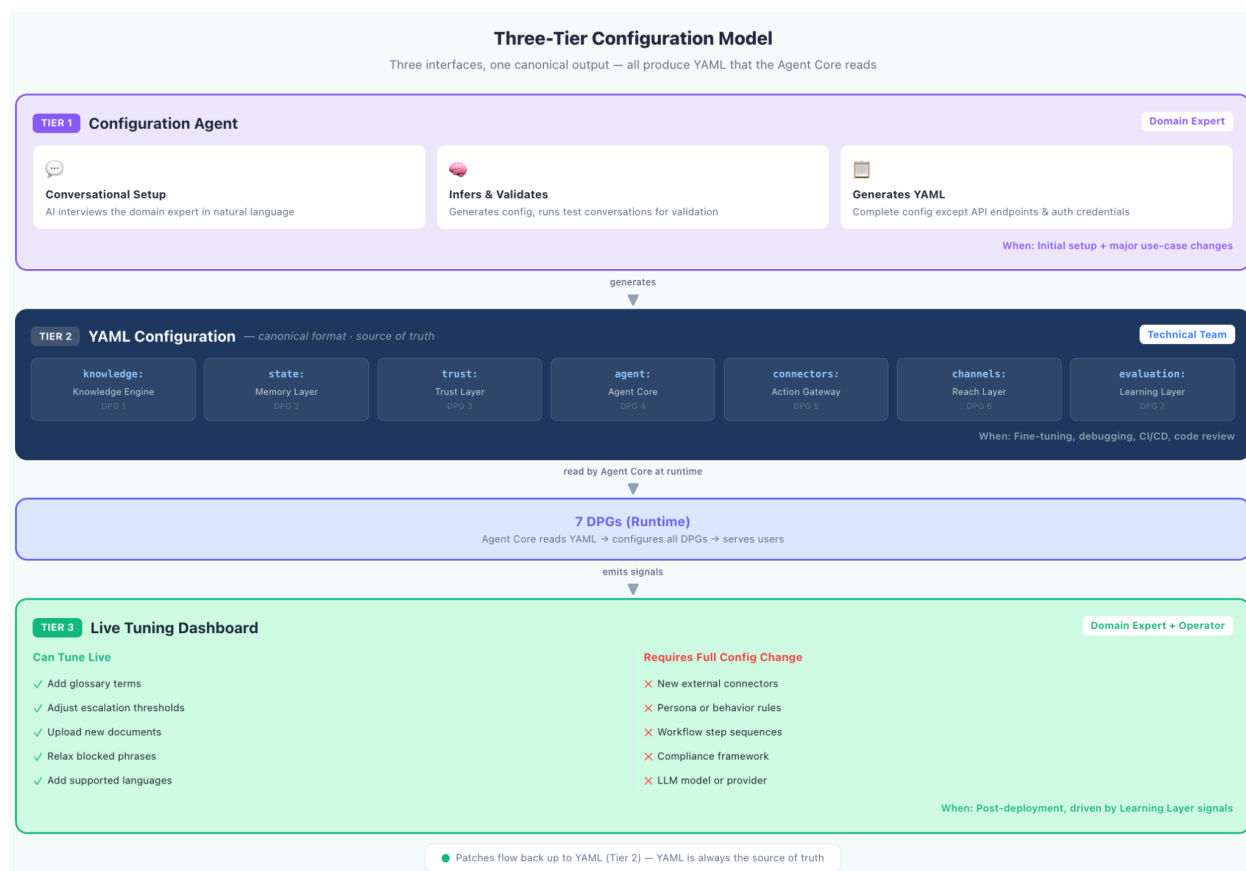
The 7 DPGs are reusable. The Domain Configuration Kit is what makes each deployment specific. The question is: **who creates the configuration, and how?**

Two realities exist simultaneously:

1. The domain expert who understands the use case (a health officer, an agriculture lead, a labour commissioner) cannot write YAML or paste API endpoints
2. The technical team who can write YAML doesn't understand the domain deeply enough to make the right configuration choices

The architecture needs to serve both — without making either dependent on the other for every change.

Three-Tier Configuration Model



All three tiers produce the same output: a structured configuration that the Agent Core reads at runtime. YAML is always the source of truth.

Tier 1: The Configuration Agent

An AI agent that interviews the domain expert across the 5 areas of the Domain Configuration Kit. It doesn't ask for YAML — it asks questions a domain expert can answer.

What the Interview Looks Like

The Configuration Agent guides the domain expert through 5 stages, each mapping to one area of the Domain Configuration Kit. It asks in natural language, validates answers, and generates structured configuration.

Stage 1 → Knowledge Corpus (configures: Knowledge Engine)

None

Agent: "What problem does this system solve for your users?"

Expert: "Farmers call to find out what crops to plant this season and how to file insurance claims when crops are damaged."

Agent: "What documents or guides should the system know about?"

Expert: "We have the crop calendar PDF, the pest management handbook, and the PMFBY insurance claim process document."

Agent: "Do your users use specific local terms for things?

For example, do they say 'fasal bima' instead of 'crop insurance'?"

Expert: "Yes – they say 'fasal bima' for insurance, 'mandi bhav' for market prices, 'keeda' for pest generally..."

Agent: "What languages do your users speak?"

Expert: "Hindi, Bhojpuri, Marathi, and some Punjabi."

What the agent generates from this:

None

Auto-generated by Configuration Agent

Stage 1: Knowledge Corpus → Knowledge Engine

knowledge:

sources:

- path: ./docs/crop_calendar_2026.pdf
type: static
refresh: seasonal
description: "Season-wise crop planting guide by district"
- path: ./docs/pest_management_handbook.pdf
type: static
refresh: annual
description: "Pest identification and treatment protocols"
- path: ./docs/pmfbby_claim_process.pdf
type: static
refresh: on_policy_change
description: "Step-by-step crop insurance claim filing"

```
glossary:
  mappings:
    - colloquial: ["fasal bima", "bima", "insurance"]
      canonical: "crop_insurance_pmfby"
    - colloquial: ["mandi bhav", "bhav", "rate"]
      canonical: "market_price"
    - colloquial: ["keeda", "keede", "pest", "kida"]
      canonical: "pest_infestation"

languages:
  supported: [hindi, bhojpuri, marathi, punjabi]
  normalization: bhashini
  code_switching: true
```

Stage 2 → Connectors & Data (configures: Action Gateway)

None

Agent: "Does the system need to check anything live –
like prices, eligibility, or availability?"

Expert: "Yes, farmers want today's mandi prices and their
PM-KISAN payment status."

Agent: "Does the system need to do anything on behalf
of the farmer – like file a form or register them?"

Expert: "Yes, it should file crop damage reports for
insurance claims."

Agent: "For filing a claim, what information do you need
from the farmer?"

Expert: "Their Aadhaar-linked phone number, crop name,
damage type, and the survey number of their land."

Agent: "Should the system verify the farmer's identity
before filing?"

Expert: "Yes, through Aadhaar OTP."

What the agent generates:

None

Stage 2: Connectors & Data → Action Gateway

connectors:

read:

- name: mandi_prices
description: "Today's market prices for farmer's crop"
endpoint: <https://api.agrimarket.gov.in/prices>
method: GET
params:
 - crop: from_conversation
 - district: from_user_profiletimeout_ms: 2000
fallback: "Mandi prices are temporarily unavailable.
Please try again shortly."

- name: pm_kisan_status
description: "Check PM-KISAN instalment status"
endpoint: <https://api.pmkisan.gov.in/status>
method: GET
params:
 - aadhaar_linked_mobile: from_user_profiletimeout_ms: 3000
requires_identity_verification: true
fallback: "PM-KISAN status check is temporarily down."

write:

- name: file_crop_damage_report
description: "File crop insurance claim under PMFBY"
endpoint: <https://api.pmfby.gov.in/claim>
method: POST
requires_consent: true
requires_identity_verification: true
fields:
 - name: crop_name
source: conversation
required: true

```

- name: damage_type
  source: conversation
  required: true
  options: [drought, flood, pest, hailstorm, other]
- name: survey_number
  source: conversation
  required: true
- name: farmer_id
  source: identity_verification
  required: true
missing_field_behavior: ask_user
success_response: "Your crop damage report has been filed.
                  Reference number: {response.ref_id}"

identity:
- name: aadhaar_otp
  provider: digilocker
  method: aadhaar_otp
  required_before: [file_crop_damage_report, pm_kisan_status]

```

Stage 3 → Conversation Design (configures: Agent Core + Knowledge Engine)

None

Agent: "How should the system talk to farmers?
Formal or informal?"

Expert: "Informal. Like a knowledgeable neighbor, not a
government officer. Simple words."

Agent: "What should the system never do?"

Expert: "Never recommend a specific pesticide brand.
Never guarantee a price. Never tell a farmer
to ignore their local KVK's advice."

Agent: "When a farmer calls, what's the most common
thing they want?"

Expert: "They want to know what's the right crop to

plant right now, or they have a pest problem and want to know what to do."

Agent: "If a farmer describes a pest problem, should the system give treatment advice directly?"

Expert: "For common known pests, yes – from our handbook. For anything unusual, it should tell the farmer to contact the KVK and give them the number."

What the agent generates:

None

Stage 3: Conversation Design → Agent Core

conversation:

persona:

name: "Kisan Mitra"

role: "Agricultural advisory helpline for smallholder farmers"

tone: "Like a knowledgeable neighbor – informal, warm, uses simple everyday language. Never bureaucratic."

behavior_rules:

- "Never recommend a specific pesticide brand name"
- "Never guarantee a crop price – always use ranges"
- "Never contradict KVK advice – defer to local KVK for anything outside the knowledge base"
- "For common pests in the handbook, give treatment advice directly with dosage from the knowledge base"
- "For unknown or unusual pest descriptions, tell the farmer to visit their KVK and provide the nearest KVK contact number"
- "Always ask which crop and which district before giving any advisory"
- "Use the farmer's name if they share it"

workflows:

- name: crop_advisory

```
trigger_intent: crop_guidance
steps:
  - ask: crop_name_if_not_known
  - ask: district_if_not_known
  - retrieve: crop_calendar_for_district_and_season
  - respond: sowing_advisory_with_source

- name: pest_identification
  trigger_intent: pest_problem
  steps:
    - ask: crop_name_if_not_known
    - ask: symptom_description
    - ask: photo_via_whatsapp_if_available
    - match: pest_from_handbook
    - if_match: respond_with_treatment
    - if_no_match: refer_to_kvkc_with_contact

- name: insurance_claim
  trigger_intent: crop_damage
  steps:
    - ask: crop_name
    - ask: damage_type
    - ask: survey_number
    - verify: aadhaar_otp
    - confirm: all_details_with_farmer
    - execute: file_crop_damage_report
    - confirm: reference_number
```

Stage 4 → Trust & Policy (configures: Trust Layer)

None

Agent: "Are there topics the system should never handle – always transfer to a human?"

Expert: "Loan recovery disputes, land ownership conflicts, and anything about subsidies being denied for political reasons."

Agent: "Should the system ask farmers for consent before storing their information?"

Expert: "Yes, always. And they should be able to ask us to delete their data."

Agent: "If a farmer sounds very distressed – maybe they've lost their entire crop – what should the system do?"

Expert: "Connect them to our helpline counsellor immediately. Don't keep them talking to AI."

What the agent generates:

None

Stage 4: Trust & Policy → Trust Layer

```
trust:
  content_safety:
    allowed_topics:
      - crop_advisory
      - pest_management
      - market_prices
      - weather_information
      - government_schemes
      - insurance_claims
    blocked_topics:
      - political_commentary
      - specific_brand_recommendations
      - land_ownership_disputes
    blocked_output_phrases:
      - "guaranteed"
      - "pakka"
      - "best pesticide"
      - "definitely get"
  pii_handling:
    detect: true
```

```
mask_in_audit: true

compliance:
  framework: DPDP_Act_2023
  consent_required_before:
    - storing_user_profile
    - filing_insurance_claim
    - checking_pm_kisan_status
  data_retention_days: 180
  right_to_deletion: true

escalation:
  to_human_when:
    - confidence_below: 0.6
    - user_requests_human: true
    - distress_detected: true
    - crop_loss_severe: true
  always_human_topics:
    - loan_recovery_disputes
    - land_ownership_conflicts
    - subsidy_denial_grievances
  human_handoff:
    method: warm_transfer
    context_passed: full_session_summary
    callback_sla_hours: 24
```

Stage 5 → Success Criteria (configures: Learning Layer)

None

Agent: "How would you know this system is working well?"

Expert: "Farmers are getting the right advisory for their crop and district. Insurance claims are actually being filed. Farmers call back – that means they trust it."

Agent: "What would tell you it's failing?"

Expert: "Wrong crop advice for the season. Farmers being told prices that don't match the mandi. Claims filed with wrong details."

What the agent generates:

None

Stage 5: Success Criteria → Learning Layer

evaluation:

primary_metric:

name: advisory_accuracy_rate

definition: "% of crop advisories that match the crop calendar
for the farmer's district and current season"

target: ">95%"

secondary_metrics:

- name: claim_completion_rate

definition: "% of insurance claim conversations that result
in a successfully filed claim"

target: ">70%"

- name: return_call_rate

definition: "% of farmers who call again within 30 days"

note: "Trust signal – farmers who return found value"

- name: price_accuracy

definition: "% of mandi prices delivered that match
actual API response"

target: ">99%"

- name: escalation_rate

target: "<15%"

note: "If above 15%, knowledge base has gaps"

quality_checks:

- name: groundedness

method: "Response supported by knowledge base or live API"

alert_threshold: "<0.8"

- name: season_match

```
method: "Crop advisory matches current season for district"  
alert_threshold: "<0.95"
```

```
drop_off_signals:
```

- point: after_advisory_delivered
re_engagement: "None – advisory delivered is success"
- point: mid_claim_filing
re_engagement: "24h callback – resume from last step"
- point: after_pest_referral_to_kvk
re_engagement: "7 days – did you visit KVK?"

Configuration Agent — Design Principles

The Configuration Agent is not a form-filling tool. It is an **expert interviewer** that:

1. **Asks in the domain expert's language.** Never ask "what are your guardrail policies?" — ask "what should the system never do?"
2. **Infers where possible.** If the expert says "farmers call about crop damage during floods," the agent infers that `damage_type: flood` is an option, that `distress_detected` should trigger escalation, and that `weather_information` is an allowed topic — without asking separately.
3. **Validates through examples.** After generating conversation design, the agent runs a simulated conversation: "Here's how the system would handle a farmer asking about wheat sowing in Madhya Pradesh. Does this look right?" The expert corrects in natural language; the agent adjusts the config.
4. **Generates complete, deployable YAML.** The output is not a draft — it is a configuration that the Agent Core can read and run. The expert can test it immediately through simulated conversations.
5. **Flags what it cannot infer.** "I need the actual API endpoint for mandi prices — your technical team will need to provide this." The agent clearly separates what the domain expert configures from what the technical team must supply.

Tier 2: The YAML Configuration (Canonical Format)

The YAML is the source of truth at runtime. It is what the Agent Core reads. Everything the Configuration Agent generates, and everything the Live Tuning Dashboard patches, ultimately becomes YAML.

Complete Configuration Structure

This is the full schema — every field maps to a specific DPG.

None

```
# =====
# USE CASE DEPLOYMENT CONFIGURATION
# Canonical format · Read by Agent Core at runtime
# =====

# — DEPLOYMENT METADATA —
deployment:
  name: string                # "Kisan Mitra" / "Kaam Ki Baat"
  version: string             # Semantic versioning
  environment: enum           # dev / staging / production
  owner: string               # Deploying organisation

# — AGENT CORE CONFIGURATION —
# Configures: Agent Core (DPG 4)
agent:
  primary_model: string       # claude-sonnet-4-5-20250514
  fallback_model: string      # claude-haiku-4-5-20251001
  temperature: float         # 0.0-1.0
  max_tokens: int             # per response
  latency_budget_ms: int      # total turn budget

# — CHANNEL CONFIGURATION —
# Configures: Reach Layer (DPG 6)
channels:
  voip:
    provider: string          # exotel / twilio
```

```

    number: string                # inbound number
    missed_call_callback: bool
  whatsapp:
    provider: string              # gupshup / twilio
    handoff_triggers: string[]    # signals that trigger voice-WA
  outbound:
    provider: string
    dnd_window: string            # "21:00-08:00"
    max_retries: int
    retry_interval_hours: int
    campaign_types: string[]      # re_engagement / alerts /
follow_through

# — KNOWLEDGE CORPUS —
# Configures: Knowledge Engine (DPG 1)
knowledge:
  sources:
    - path: string                # file path or URL
      type: enum                  # static / semi_static / dynamic
      refresh: enum                # manual / daily / weekly / monthly
      description: string          # what this source covers
  glossary:
    mappings:
      - colloquial: string[]       # user terms
        canonical: string          # system term
  languages:
    supported: string[]            # [hindi, english, kannada]
    normalization: string         # bhashini / custom
    code_switching: bool
    transliteration: bool

# — CONNECTORS & DATA —
# Configures: Action Gateway (DPG 5)
connectors:
  read:
    - name: string                # unique connector ID
      description: string
      endpoint: string            # API URL

```

```

    method: enum                # GET / POST
    auth:
      type: enum                # api_key / oauth / none
      key_env: string           # environment variable name
    params:
      [param_name]:
        source: enum            # conversation / user_profile /
static
  timeout_ms: int
  retry: int
  fallback: string              # response when API fails
  requires_identity_verification: bool

write:
  - name: string
    description: string
    endpoint: string
    method: enum
    auth: object
    requires_consent: bool      # must be true for write connectors
    requires_identity_verification: bool
    fields:
      - name: string
        source: enum            # conversation / user_profile /
identity
  required: bool
  options: string[]            # constrained values (optional)
  missing_field_behavior: enum  # ask_user / skip / fail
  success_response: string      # template with {response.*} vars
  failure_response: string

identity:
  - name: string
    provider: string            # digilocker / custom
    method: string              # aadhaar_otp / service_number
    required_before: string[]    # list of connector names

```

— CONVERSATION DESIGN —

```

# Configures: Agent Core (DPG 4) + Knowledge Engine (DPG 1)
conversation:
  persona:
    name: string
    role: string          # one-line role description
    tone: string          # natural language tone guide

  behavior_rules: string[]  # list of do/don't rules

  user_mental_model:      # optional - for complex journeys
    states:
      - name: string
        description: string
        system_behavior: string

  workflows:
    - name: string
      trigger_intent: string
      steps:
        - action: enum      # ask / retrieve / match / verify /
                           # confirm / execute / respond /

  refer
    target: string          # field name / connector name /

  message
    condition: string       # optional - if_match / if_no_match

# — TRUST & POLICY —
# Configures: Trust Layer (DPG 3)
trust:
  content_safety:
    allowed_topics: string[]
    blocked_topics: string[]
    blocked_output_phrases: string[]
  pii_handling:
    detect: bool
    mask_in_audit: bool

  compliance:

```

```

    framework: string          # DPDP_Act_2023 / GDPR / custom
    consent_required_before: string[]
    data_retention_days: int
    right_to_deletion: bool

escalation:
  to_human_when:
    confidence_below: float
    user_requests_human: bool
    distress_detected: bool
    custom_triggers: string[]  # domain-specific triggers
  always_human_topics: string[]
  human_handoff:
    method: enum              # warm_transfer / cold_transfer
    context_passed: enum      # full_session / summary / none
    callback_sla_hours: int

# — SUCCESS CRITERIA & EVALUATION —
# Configures: Learning Layer (DPG 7)
evaluation:
  primary_metric:
    name: string
    definition: string
    target: string

  secondary_metrics:
    - name: string
      definition: string
      target: string
      note: string           # optional context

  quality_checks:
    - name: string
      method: string
      frequency: enum        # every_turn / end_of_session /
daily
  alert_threshold: string

```

```

drop_off_signals:
  - point: string          # where in the journey
    re_engagement: string  # action to take

# — STATE CONFIGURATION —
# Configures: Memory Layer (DPG 2)
state:
  session_store: string    # redis://...
  profile_store: string    # postgres://...
  retention_days: int
  broadcast:
    enabled: bool
    channel: string        # pub/sub topic

# — OBSERVABILITY —
# Configures: Learning Layer (DPG 7)
observability:
  audit_level: enum        # full / summary / minimal
  log_sink: string         # endpoint for log ingestion
  pii_masking: bool
  alert_channels: string[] # email / slack / webhook

```

How the YAML Maps to DPGs

None

YAML Section	→ DPG Configured
deployment	→ Metadata (all DPGs)
agent	→ Agent Core
channels	→ Reach Layer
knowledge	→ Knowledge Engine
connectors	→ Action Gateway
conversation	→ Agent Core + Knowledge Engine
trust	→ Trust Layer
evaluation	→ Learning Layer
state	→ Memory Layer
observability	→ Learning Layer

Tier 3: Live Tuning Dashboard

Post-deployment, the Learning Layer continuously surfaces signals. The Live Tuning Dashboard lets domain experts and operators make targeted adjustments without touching the full YAML.

What Can Be Tuned Live

Signal from Learning Layer	Tuning Action	YAML Section Affected
Low groundedness on topic X	Upload new document or FAQ	<code>knowledge.sources</code>
Users saying term not in glossary	Add glossary mapping	<code>knowledge.glossary</code>
High escalation rate on intent Y	Adjust confidence threshold or add knowledge	<code>trust.escalation / knowledge</code>
Connector timeout rate > threshold	Adjust timeout or update fallback	<code>connectors.*.timeout_ms / fallback</code>
Users asking about new topic	Add to allowed topics + upload knowledge	<code>trust.content_safety + knowledge</code>
Post-action mismatch reports	Flag connector data quality	Connector alert (not config change)
Guardrail blocking legitimate queries	Remove or relax blocked phrase	<code>trust.content_safety.blocked_output_phrases</code>
New language/dialect appearing	Add to supported languages	<code>knowledge.languages</code>

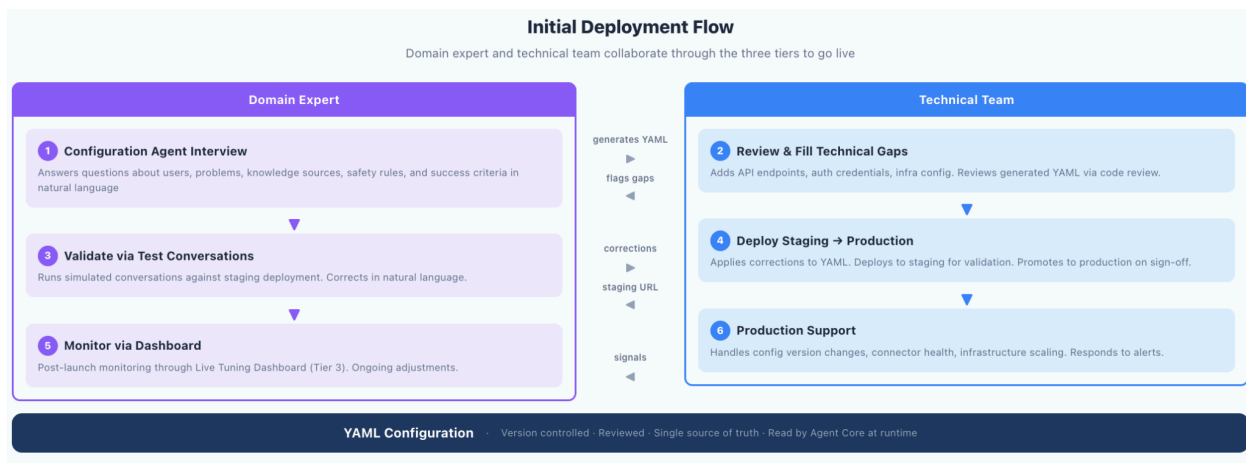
What Cannot Be Tuned Live (Requires Full Config Change)

- Adding new connectors (new external systems)
- Changing the core persona or behavior rules
- Modifying workflow step sequences
- Changing compliance framework
- Switching LLM model or provider

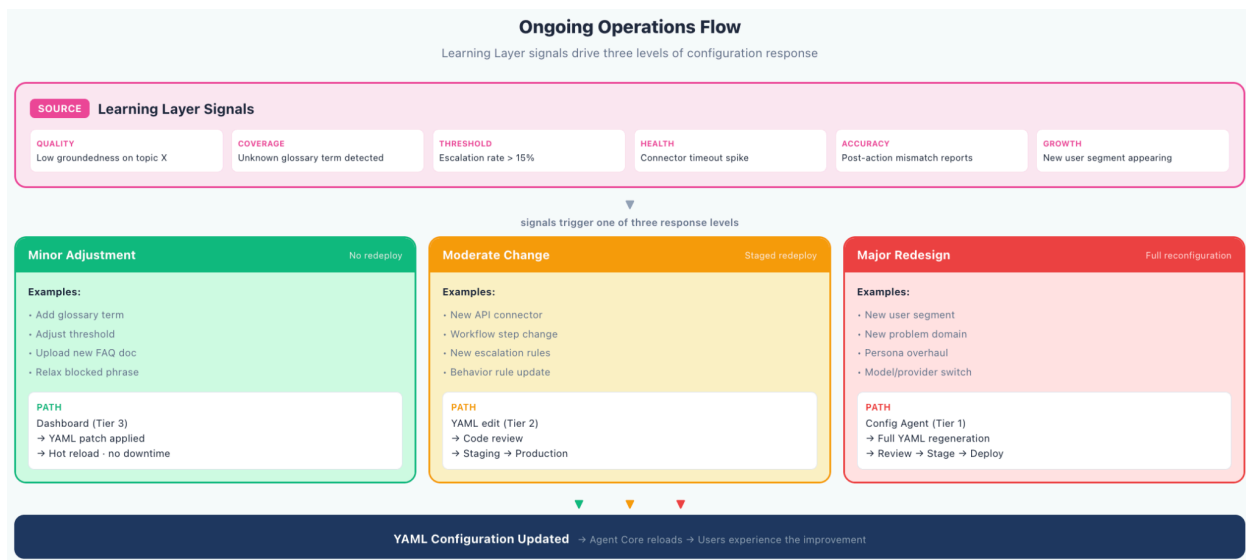
These require a config version change, review, and redeployment — handled through Tier 2.

How the Three Tiers Work Together

Initial Deployment



Ongoing Operations

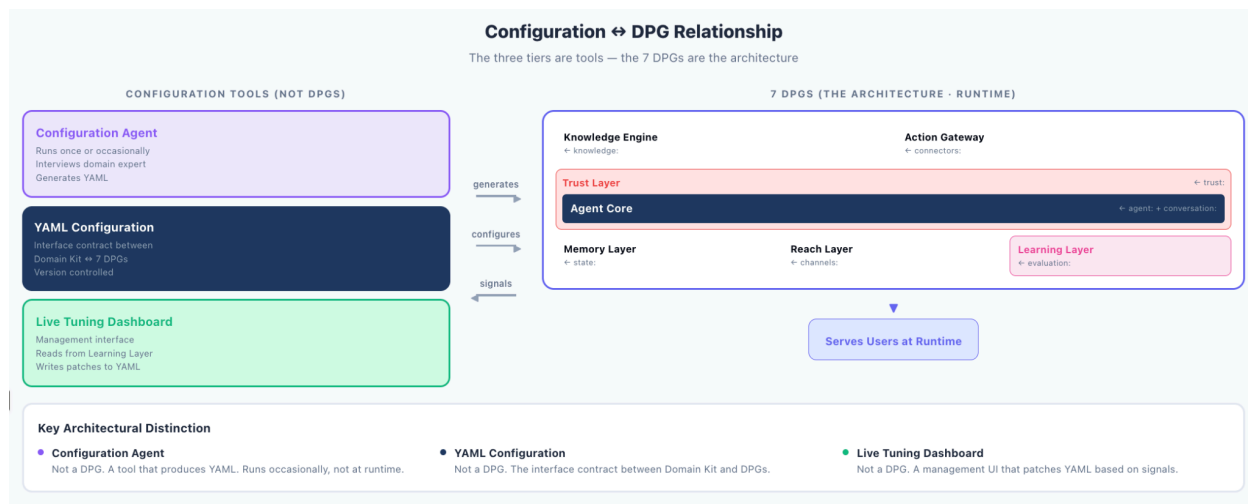


What This Means for the DPG Architecture

The Configuration Agent is **not an 8th DPG**. It is a tool that produces configuration for the 7 DPGs. It runs once (or occasionally) — not at runtime. It could itself be built on the same composition framework (it's an AI agent that interviews users and generates structured output), but it is not part of the deployed use-case system.

The YAML configuration is not a DPG either — it is the **interface contract** between the Domain Configuration Kit and the 7 DPGs. Every DPG reads its behavior from the relevant YAML section.

The Live Tuning Dashboard is a **management interface** — it reads from the Learning Layer and writes patches to the YAML. It is operational tooling, not a runtime component.



The 7 DPGs remain the architecture. The three-tier configuration model is how they get configured — not how they run.

Sample Use Case (Detailed)

Kaam Ki Baat

Product & Architecture Document

Dial 5226. Speak your language. See the market clearly. Then decide.

Version 1.0 · March 2026

PART 1 — THE KAAM KI BAAT EXPERIENCE

1. The User Universe
2. Entity Map — What the System Knows
3. The Five Mental States — Fog to Follow-Through
4. Decision Parameters by Persona
5. ITI Graduate — Full Conversation Tree
6. Scenario Library — Positive, Negative, Re-engagement
7. Drop-off Taxonomy & Success Metrics

PART 2 — BUILDING KKB ON DPG BLOCKS

8. Block Selection & Rationale
9. Multimodal Continuity — Call ↔ WhatsApp
10. Memory Architecture
11. ONEST Integration via Dynamic Data Connector
12. KKB System Wiring — DPG Block Diagram
13. Configuration Sketch

1. The User Universe

KKB serves workers who face information asymmetry — they cannot see the labour market clearly, and this invisibility costs them time, money, and dignity.

Persona	Profile	Core Anxiety	Primary Constraint
ITI Graduate ★	19–24, trade-certified, first job seeker	"Is my skill worth anything here?"	Distance + skill confidence
Women Returning to Work	26–38, career gap of 2–8 years	"Has the market moved on without me?"	Hours + distance + family approval
Daily Wage Labourer	30–45, informal, no fixed employer	"Is today going to be a wasted morning?"	Immediacy + daily income certainty
AI-Displaced Worker	35–50, formal sector, job eliminated	"Am I obsolete?"	Income continuity + dignity
Person with Disability	Any age, specific accessibility needs	"Will I be pitied or employed?"	Role accessibility + remote options

★ Primary persona for this document. Others are referenced at branch points where journeys diverge.

2. Entity Map — What the System Knows

Every user in KKB has five layers of profile data. The system builds this progressively — never asking for everything upfront, never forgetting what it already knows.

IDENTITY LAYER — Who they are
• user_id — phone number as primary key
• name — optional, used for personalisation
• age_bracket — 19–24 / 25–34 / 35–45 / 45+
• gender
• disability_status — none / mobility / visual / hearing / cognitive
• language_preference — Hindi / Kannada / English / other
• location — district + pin zone, not full address
CAPABILITY LAYER — What they have
• education_level — below 10th / 10th / 12th / ITI / diploma / graduate
• trade_or_stream — electrician / fitter / welder / accounting / BPO / etc.

• certifications — NCVT / PMKVY / sector-specific
• years_experience — 0 / <1 / 1–3 / 3–7 / 7+
• informal_skills — family business, self-taught, etc.
• language_skills — read/write vs speak-only
• digital_literacy — none / basic / smartphone / advanced

CONSTRAINT LAYER — *What limits their choices*

• max_commute_km — 2 / 5 / 10 / 20 / city-wide / remote-only
• income_urgency — immediate <30 days / short-term 1–3 months / flexible
• availability_hours — full-time / part-time / school-hours only / night shift
• mobility_owned — none / bicycle / two-wheeler / car
• dependents — none / children / elderly / both

ASPIRATION LAYER — *What they want (may differ from what they ask)*

• primary_goal — stable income / career growth / flexibility / dignity / any income
• sector_preference — manufacturing / services / govt / remote / no preference
• growth_horizon — immediate / 1 year / 3 years
• open_to_training — yes / only if short / no

JOURNEY LAYER — *Where they are in the process*

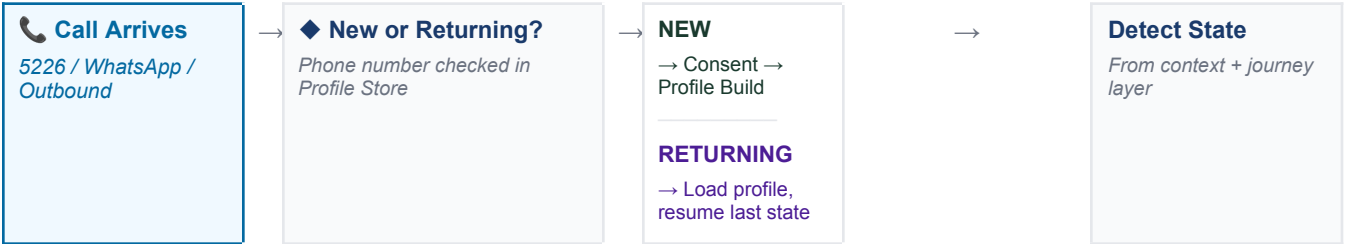
• session_count
• current_mental_state — fog / orientation / evaluation / commitment / follow-through
• last_market_truth_delivered — timestamp + summary
• options_presented — with timestamps
• actions_taken — applied / enrolled / counsellor_connected
• outcomes_tracked — callback received / joined / left / unknown
• drop_off_reason — if applicable

Key design rule: The system never asks for data it can infer. If the user already said they are in Hubli and want work within 5km, it never asks again — even across sessions, even across channels.

3. The Five Mental States — Fog to Follow-Through

A caller is never just 'looking for a job.' They are in one of five distinct mental states. Detecting the correct state — and responding to it appropriately — is the system's primary intelligence task.

State Flow Diagram



1 FOG <i>"Koi kaam hai kya?"</i> Does not know what they want. System must not jump to options. Deliver market truth first.	2 ORIENTATION <i>Asking about pay, skills, training</i> Understands the landscape. System shifts to decision support. Fill entity map gaps.	3 EVALUATION <i>Comparing real options</i> Highest-value state. System surfaces decision parameters. Never pushes, always honest trade-offs.	4 COMMITMENT <i>"Apply kar do"</i> User has decided. System removes friction. Consent mandatory before every action.	5 FOLLOW-THROUGH <i>Post-action check-in</i> Did employer call? Did course start? Did job match? Trust is built or broken here.
--	--	---	---	--

- State 1 — FOG:** Does not know what they want. System must not jump to options. Deliver market truth first.
- State 2 — ORIENTATION:** Understands the landscape. System shifts to decision support. Fill entity map gaps.
- State 3 — EVALUATION:** Highest-value state. System surfaces decision parameters. Never pushes, always honest trade-offs.
- State 4 — COMMITMENT:** User has decided. System removes friction. Consent mandatory before every action.
- State 5 — FOLLOW-THROUGH:** Did employer call? Did course start? Did job match? Trust is built or broken here.

4. Decision Parameters by Persona

Decision parameters are not universal. The same question — 'should I take this job?' — has a different answer tree depending on who is asking.

ITI Graduate (Primary Persona)

Rank	Parameter	Why it matters for this persona
1	Distance from home	First job, no owned vehicle likely, family may restrict range
2	Skill match confidence	"Can I actually do this?" — fear of failure in first role
3	Income certainty	Stability > amount. First salary is a milestone, not just money
4	Growth trajectory	Is this a dead end or a stepping stone? Strongly weighted for youth
5	Time to first income	If family is dependent, a 3-month course may not be viable
6	Social acceptability	Sector, employer type, and peer perception matter at this age
7	Dignity of work	Less acute than other personas but present — formal vs informal matters

Women Returning to Work

Rank	Parameter	Why it matters for this persona
1	Availability hours	School hours, home obligations — this is often non-negotiable
2	Distance	Safety perception + family approval of commute
3	Skill gap size	6-year gap creates anxiety — "what do I not know?"
4	Income certainty	Supplementary income vs primary — context-dependent
5	Social acceptability	Type of employer, co-worker demographics, sector norms
6	Growth trajectory	Lower weight — re-entry stability often prioritised over growth
7	Dignity of work	High — not willing to settle for work that feels beneath prior career

Daily Wage Labourer

Rank	Parameter	Why it matters for this persona
1	Time to first income	Today, not next week. Immediate is the only timeframe
2	Distance	Walk-able or one bus stop — no float for commute cost
3	Income certainty	Daily guaranteed > higher but uncertain
4	Dignity of work	Bypassing the chowk crowd is itself a dignity gain
5	Skill match confidence	Lower anxiety — broad skill set, flexible
6	Growth trajectory	Not weighted heavily — survival mode
7	Social acceptability	Minimal relevance

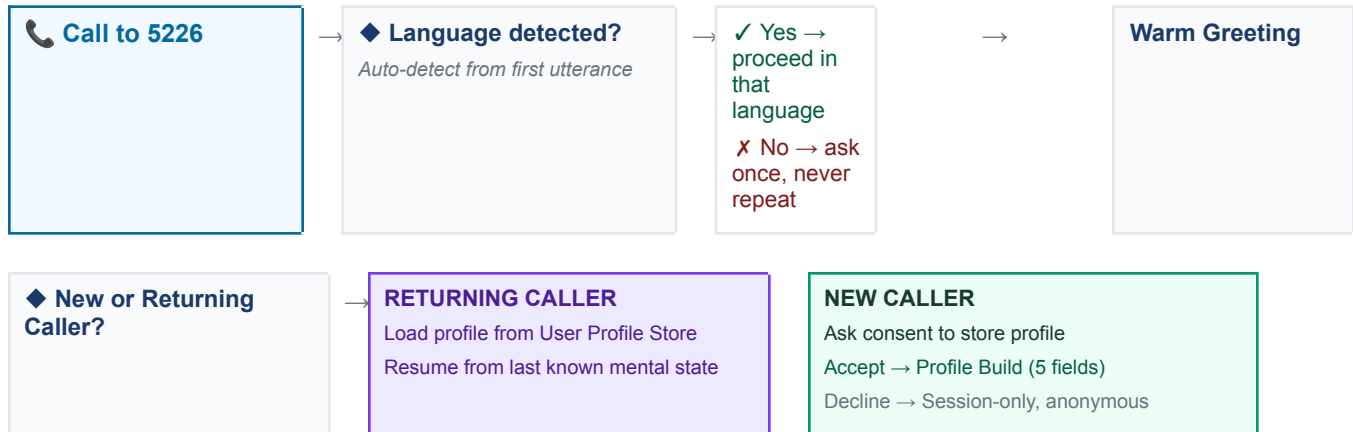
AI-Displaced Worker

Rank	Parameter	Why it matters for this persona
1	Income continuity	Had stable salary — gap is terrifying, not just inconvenient
2	Dignity of work	Has had a career. Will not accept work that feels like regression
3	Skill match confidence	Deep anxiety — 20 years of experience may feel suddenly worthless
4	Time to first income	Moderate urgency — some savings but finite runway
5	Growth trajectory	"Is there a future here?" matters more than "will I grow fast?"
6	Distance	More flexible — experienced, may have vehicle
7	Social acceptability	Moderate — does not want to explain a step-down publicly

5. ITI Graduate — Full Conversation Tree

6 primary branches determined by two key variables: Market signal strength (strong / weak / absent) and Skill match level (direct / partial / gap exists). Every node has: a system action, a user response category, and a branch condition.

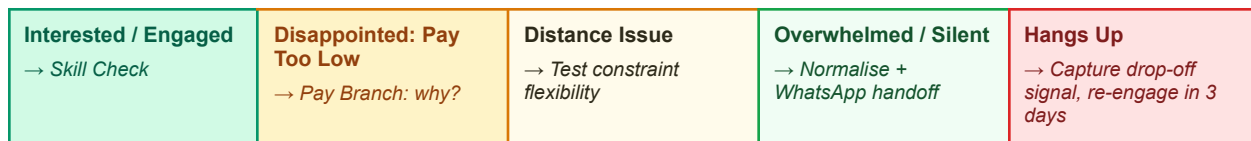
Conversation Flow Diagram



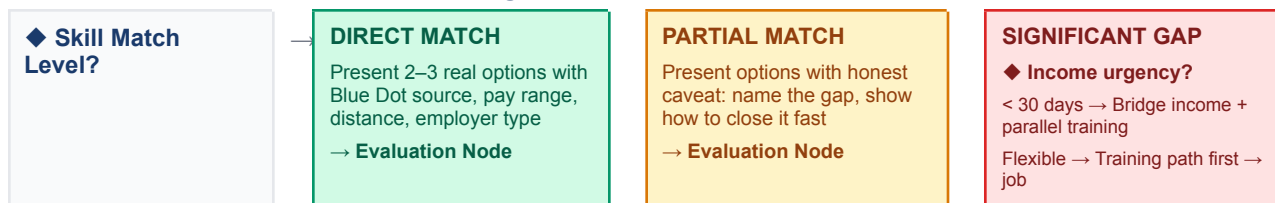
MARKET TRUTH DELIVERY

What work exists nearby · What pay is realistic (as range, never absolute)
How many roles approximately · What skills are actually needed
Source: *ONEST Blue Dot* — verified, location-tagged, time-stamped

User Reaction to Market Truth — 5 Possible Branches:

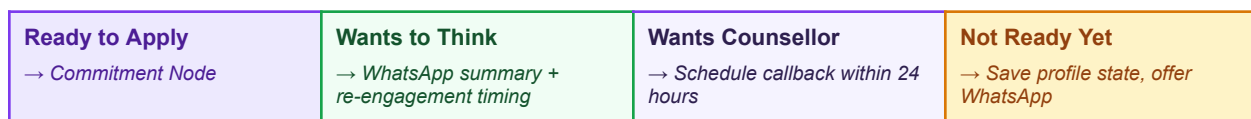


Branch: Skill Check → Gap Handling



EVALUATION NODE

User comparing options. System presents honest trade-offs. Never pushes one path.



COMMITMENT NODE

Confirm consent: 'Kya main apply kar doon?' · Confirm specific role

Apply for Job

Submit application via ONEST Blue Dot.
Confirm reference number to user.

Enrol in Course

Connect to institute or PMKVY portal.
Confirm seat availability.

Connect Counsellor

Schedule callback from career
counsellor within 24 hours.

Set Follow-up

72h for job callback check · 1 week for course start · User
consent for re-contact



FOLLOW-THROUGH CHECK

Did employer call? · Did course start? · Is job as
described?

Positive

Record success, capture
learnings, offer next step if
needed

Employer Ghost

Offer next option from original
shortlisted roles

Job Mismatch

Re-enter journey at State 2
with full prior context. Flag Blue
Dot source.

No Response

Mark outcome unknown.
Passive re-engagement at 30
days.

Branch Condition Reference Table

Node	Condition	Branch Taken	System Behaviour
Language detection	From first utterance	Proceed in detected language	Auto-switch, no interruption
Language detection	Ambiguous	Ask once, accept any response	Never ask twice
New / Returning	Phone number match	Load full profile	Resume from last known state
New / Returning	No match	Start profile build	Consent first
Consent	Declined	Anonymous session	No profile stored, session-only memory
Market truth reaction	Positive engagement	Skill check	Proceed to options
Market truth reaction	Pay disappointment	Explore why	Realign or acknowledge honestly
Market truth reaction	Distance issue	Test constraint flexibility	Show local-only if hard constraint
Market truth reaction	Overwhelmed / silent	Normalise, offer WhatsApp	Never push through discomfort
Market truth reaction	Hang-up	Capture drop-off signal	Re-engage at 3 days
Skill match	Direct match	Present options immediately	2–3 options, Blue Dot sourced
Skill match	Partial	Present with honest caveat	Name the gap, show close-fast options
Skill match	Significant gap	Route by income urgency	Bridge income OR training path
Income urgency + gap	< 30 days	Bridge income + parallel training	Both paths shown, user chooses
Income urgency + gap	Flexible	Training path first	Course → job sequence

Evaluation	Ready to apply	Action node	Consent confirmation mandatory
Evaluation	Wants to think	WhatsApp handoff	Summary sent, no pressure
Evaluation	Wants counsellor	Schedule callback	Within 24 hours
Evaluation	Not ready	Save and return	Profile saved, re-engagement offered
Action type	Apply for job	Submit via ONEST	Confirm reference number
Action type	Enrol in course	PMKVY / institute connect	Confirm seat
Market signal	No local roles exist	Honest scarcity response	Alert signup, ONEST signal captured
Follow-through	Employer ghosted	Offer next option	From original shortlisted roles
Follow-through	Job mismatch	Re-enter journey at State 2	Full prior context preserved

6. Scenario Library — Positive, Negative, Re-engagement

Nine scenarios trace how different journeys play out — what succeeds, what drops off, and what comes back.

Positive Scenarios

P1 — Single-Call Placement

Arjun, 21, ITI electrician, Hubli. First call. Strong ONEST signal: 12 verified electrician roles within 6km, pay ₹14,000–18,000/month. Direct skill match. Evaluates 2 options on the call. Applies to one. Employer callback within 48 hours. System follows up at 72h, confirms callback received. Session closed as success.

Drop-off risk: None · Success signal: *Application → employer callback → user confirmation*

P2 — Training-Then-Placement

Meena, 23, ITI electronics, Bangalore outskirts. Gap: operator certification required (3-week course). Income urgency: moderate. System presents course + bridge income path. Meena enrolls. WhatsApp reminder at week 2. Re-call at week 3. Applies at course completion. Placed in 10 days.

Drop-off risk: Mid-training · Success signal: *Enrolment → course completion check-in → placement*

P3 — Multimodal Journey (Call → WhatsApp → Call)

Ravi, 22, ITI fitter, Pune. Market truth delivered, 3 options shown. Overwhelmed — system offers WhatsApp summary. Ravi reads over 2 days. Replies: "pehla wala theek lagta hai." System asks if he wants to apply — he says "call karo." Callback triggered. Picks up from exactly where he left off. Applies. Placed.

Drop-off risk: Between call and WhatsApp reply · Success signal: *WhatsApp reply → re-call → application*

P4 — Lateral Career Discovery

Deepak, 20, ITI electrician, Hyderabad. Asks about traditional wiring. ONEST: crowded market (40+ supply, 8 demand). System delivers honestly. Pivots to EV charging — growing 200% YoY, PMKVY course available, 6 weeks, stipend. Deepak enrolls. Placed 8 weeks later.

Drop-off risk: At market truth disappointment · Success signal: *Pivot acceptance → PMKVY enrolment*

P5 — Counsellor Referral That Converts

Sunita, 24, ITI COPA, Jaipur. Evaluated 3 options across 2 calls, cannot decide. System detects repeated evaluation loop (3rd session, no action). Flags for counsellor. Callback happens. She takes a role she had previously seen but was anxious about. Placed.

Drop-off risk: Decision paralysis loop · Success signal: *Counsellor handoff → placement*

Negative / Drop-off Scenarios

N1 — Market Gap — No Jobs Exist

Suresh, 22, ITI welder, small district in UP. ONEST signal: 1 role, 35km away, pay below expectation. System delivers honestly, no false comfort. Offers alert when new roles appear. Suresh declines. System captures: drop_off_reason: market_gap, location, trade. Dashboard signal: policy-level demand gap.

Dashboard signal: *Location-trade demand gap → feeds ONEST data intelligence*

N2 — Skill Gap Too Large, Income Urgency Too High

Kavya, 23, ITI electronics, Nagpur. Wants PCB testing. Gap: 6-month course required. Income urgency: immediate. System says explicitly: "Abhi ke liye kaam paana mushkil hoga." Presents bridge income options. Kavya disappointed but understands. Profile saved. Re-engagement at 3 months.

Dashboard signal: *Skill-income urgency mismatch → count by district → policy signal*

N3 — Application Sent, Employer Ghost

Rahul, 21, ITI electrician, Lucknow. Applied via KKB. No employer callback at 72 hours. System detects and re-contacts. Offers: next option from shortlist, or re-run market check. Rahul picks next option. This one calls back. Placed.

Dashboard signal: *Employer callback rate by Blue Dot source → low-callback sources flagged*

N4 — Job Mismatch — Role Not As Described

Anand, 22, ITI fitter, Surat. Calls back after 2 weeks: "kaam alag hai, pay bhi thoda kam hai." System does not dismiss. Captures mismatch details. Flags Blue Dot source for ONEST review. Re-opens journey at State 2 with full prior context. Applies to next option.

Dashboard signal: *Post-placement mismatch rate by source → critical ONEST quality metric*

N5 — Repeated Non-Conversion — Decision Paralysis

Priya, 23, ITI electronics, Pune. Called 4 times, evaluates carefully, never applies. System flags loop_count: 4. Auto-offers counsellor referral. If declined, WhatsApp check-in: 'Kya koi cheez rok rahi hai?' Captures qualitative reason.

Dashboard signal: *Evaluation loop count → counsellor capacity planning*

N6 — Mid-Course Dropout

Vikram, 21, ITI COPA, Delhi. Enrolled in PMKVY, stops attending week 2. System detects via ONEST institute signal. WhatsApp check-in: 'Kaam mila kya? Ya kuch aur hua?' Non-judgmental. If no response: two check-ins max, then marks outcome: unknown, enrolled_dropout. Does not spam.

Dashboard signal: *Course dropout rate by type and location → training quality intelligence*

Re-engagement Scenarios

R1 — Seasonal Re-entry

Daily wage labourer, no local work in monsoon. KKB June: 'kaam kam hai.' Alert saved. October: ONEST shows construction surge. Outbound call: 'Aapne June mein kaam dhundha tha — nayi opportunities aayi hain.' Placed within 1 call.

R2 — Post-Training Re-entry

Told 3 months ago: 'pehle course karo.' Did not engage during training. Calls back: 'course ho gaya.' Profile loaded, trade upgrade confirmed, market re-run with new qualification. Journey resumes at Orientation with significantly better signal.

R3 — Life Event Re-entry

Women returning to work. Had called when children were young. Calls back — children older, more hours available. System detects availability_hours may have changed. Asks gently: 'Pichli baar school timing ke baad kaam chahiye tha — abhi bhi waisa hi hai?' Updates constraint. Re-runs market search. Often opens significantly more options.

7. Drop-off Taxonomy & Success Metrics

Every drop-off captures a reason signal — not just a drop event. Every metric is user-centric, not conversion-obsessed.

Drop-off Taxonomy

Drop-off Point	Code	Capture Method	Re-engagement Trigger
After market truth delivery	DOP_MT	Last utterance + sentiment	3 days — 'Koi sawaal tha?'
After option presentation	DOP_OP	Which option was last discussed	5 days — WhatsApp summary resend
Mid evaluation loop	DOP_EV	Loop count + options evaluated	7 days — counsellor offer
After WhatsApp handoff (no reply)	DOP_WA	WhatsApp delivery confirmed, no reply	3 days — follow-up message
Post-application (employer ghost)	DOP_EG	72h no callback detected	Auto — offer next option
Mid-course dropout	DOP_CD	Institute signal or absence thereof	14 days — non-judgmental check-in
Market scarcity (no options)	DOP_MS	Location + trade pair captured	Alert when ONEST signal appears
Skill-income mismatch	DOP_SI	Skill gap size + urgency level	90 days — market re-check
Repeated no-action loop	DOP_RL	Loop count ≥ 3	Counsellor referral offer

Success Metrics

Primary	Informed Decision Rate % of callers who reach State 3 (Evaluation) or beyond, regardless of whether they apply. A user who understood the market and chose not to apply is a success. A user who applied without understanding is not.
Secondary	Days to First Relevant Option Median sessions and calendar days between first call and first genuine-match option presented. Target: ≤ 1 session for strong ONEST signal. ≤ 2 sessions for skill gap cases.
Tertiary	Placement Rate by Path Single call / multimodal / training-then-placement / counsellor-assisted — broken down separately. Aggregation hides which path works for whom.
Quality	False Positive Rate

% of presented options that turned out to be unavailable, inaccurate, or misrepresented. Target: < 2%. This is the core trust metric. One false promise destroys the brand.

Engagement
Return Call Rate
% of callers who call again within 30 days. Users who return have found value even without placing yet. Stronger signal than single-session conversion.

Ecosystem
ONEST Signal Contribution
Count of drop-off events captured as demand gap signals fed back to ONEST. KKB is not just a consumer of ONEST data — it is a contributor. Every 'koi kaam nahi hai yahan' is a data point.

Watch
Counsellor Overflow Rate
% of sessions ending in counsellor referral without resolution. If this rises above 15%, the conversation tree has gaps or ONEST signal quality needs attention.

8. Block Selection & Rationale

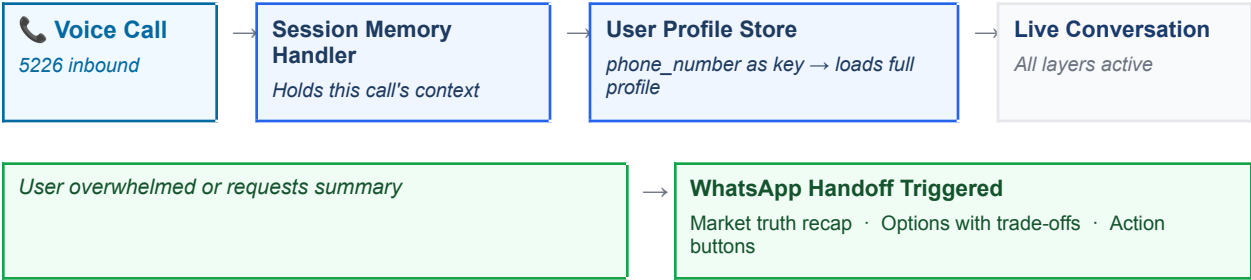
KKB requires 15 of the 25 DPG blocks. Each is justified by a specific KKB requirement — no block is included speculatively.

Block	Tier	Why KKB Needs It
Static Knowledge Base	T1	Stores labour scheme information (PMKVY, NAPS, apprenticeship rules), trade descriptions, and training institute catalogue — content that changes slowly
Dynamic Data Connector	T1	Queries ONEST Blue Dot API in real time — live job demand by location, trade, pay range. This is the heart of market truth delivery
Session Memory Handler	T1	Holds conversation state within a call — what has been discussed, which options presented, what constraints confirmed
User Profile Store	T1	Cross-session persistent memory — the entity map. Without this, multimodal continuity and re-engagement are impossible
Task State Manager	T1	Manages multi-step flows: profile build → market truth → evaluation → action. Also manages mid-course dropout detection and follow-through tracking
NLU Processor	T2	Extracts intent and entities from natural speech: trade name, location mention, constraint expressed, emotional tone. KKB conversations are unstructured — users speak freely
Language Normalization Layer	T2	Hindi-English code switching, Kannada, regional dialects — workers do not speak textbook Hindi. Bhashini wrapper is the backbone
Guardrails Engine	T3	Enforces KKB brand DNA at output level — blocks any response containing guarantee language, over-promises, or unsourced claims. Non-negotiable
HITL Handler	T3	Triggers counsellor referral when loop count ≥ 3 , when user expresses distress, or when user explicitly requests human support
LLM Inferencing Wrapper	T4	Generates the actual response — market truth framing, trade-off articulation, normalising statements. Operates under strict KKB guardrails
Manager Agent	T4	Routes each turn to the right block. 'Kaam chahiye' → Dynamic Data Connector. 'Course kitne din ka' → Static KB. 'Apply kar do' → Actions Executor
Campaign Orchestrator	T4	Drives all re-engagement: 3-day drop-off callbacks, 72-hour employer ghost follow-ups, seasonal ONEST alert outbound calls
Actions Executor	T5	Executes commitment: job application to ONEST, PMKVY enrolment, counsellor callback scheduling. Always gated by user consent confirmation
LLM Observability	T6	Every session logged — critical for measuring false positive rate, counsellor overflow, and ONEST signal quality
Feedback Signal Collector	T6	Captures job mismatch and employer ghost signals as structured feedback into ONEST quality pipeline

9. Multimodal Continuity — Call ↔ WhatsApp

The user should never feel they have started over. The phone number is the universal key. Full entity map and journey state travel with the user across every channel switch.

Channel Transition Flow



User actions on WhatsApp — 3 paths:

Replies with choice Actions Executor processes reply → ONEST	Requests callback Campaign Orchestrator → outbound call → session resumed seamlessly	No reply in 3 days One re-engagement message sent. Drop-off captured if still no response.
--	--	--

Handoff Rules

Direction	Triggered By	What Transfers	What Does NOT Transfer
Call → WhatsApp	User overwhelm signal, user request, or evaluation loop ≥ 2 without action	Market truth recap in plain language, 2–3 option cards (title, pay range, distance, one-line trade-off), action button per option, 'Call me back' button	Anything requiring an immediate decision. WhatsApp is a thinking space, not a closing mechanism.
WhatsApp → Call	User replies requesting callback, or selects 'Call me back' button	Full profile state, WhatsApp thread summary, last user statement	Fresh greeting. Call picks up from last user statement: 'Aapne pehla option choose kiya tha — kya abhi apply karen?''

10. Memory Architecture

KKB's memory has four layers, each with a different lifespan, scope, and purpose.

Layer 1 — In-Session Memory · Block: Session Memory Handler			
Lifespan Current call only	Stores Turn-by-turn dialogue, options discussed, constraints confirmed this session	Used for Context for each LLM call within the session	Cleared when On call end (unless profile consent given)

Layer 2 — Profile Memory · Block: User Profile Store			
Lifespan	Stores	Used for	Cleared when

Indefinite (until user requests deletion)	Full entity map — identity, capability, constraints, aspirations. Key: phone_number	Multimodal continuity, re-engagement, personalisation across all future sessions	Never, unless user explicitly requests. Updates on new confirmed information.
---	---	--	---

Layer 3 — Journey Memory
Block: Task State Manager

Lifespan Per journey (resets on successful placement)	Stores Current mental state, options presented, actions taken, follow-up schedule, drop-off reasons	Used for Re-engagement targeting, multimodal continuity, counsellor handoff context	Cleared when On successful placement + 30 days. Archived, not deleted.
---	---	---	--

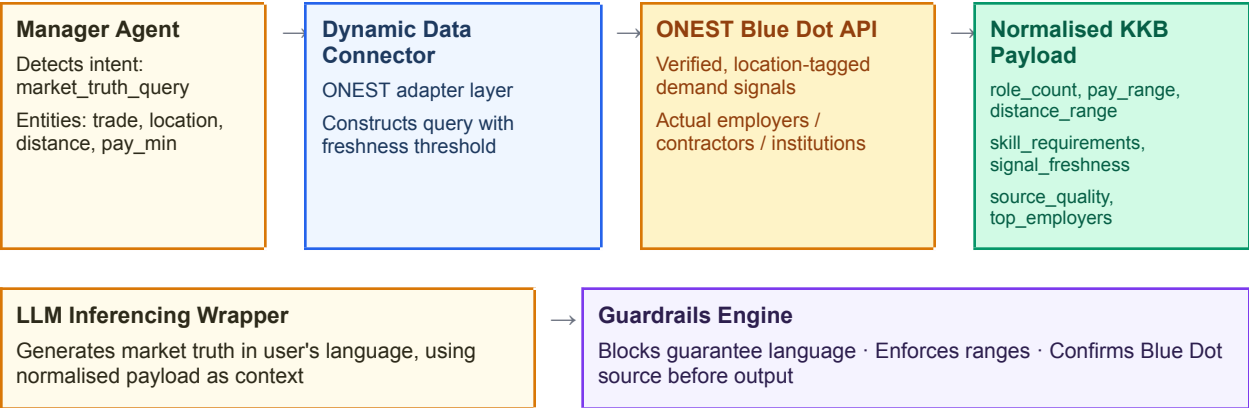
Layer 4 — Signal Memory
Block: Feedback Signal Collector

Lifespan Permanent (anonymised after 90 days)	Stores Drop-off events, mismatch signals, employer ghost events, market gap observations	Used for ONEST quality feedback pipeline, dashboard intelligence, policy signals	Cleared when Never deleted. PII stripped at 90 days.
---	--	--	--

Memory update rules: New information always overwrites old. Conflicting signals are flagged, not silently resolved. Sensitive fields (location precision, disability status) stored only at the level the user has explicitly shared.

11. ONEST Integration via Dynamic Data Connector

ONEST Blue Dots are the single most important data source in KKB. Every market truth delivered must trace back to a verified, location-tagged Blue Dot signal.



- Scarcity Handling — When ONEST Returns Zero or Very Few Results:**
 - System does NOT hallucinate roles
 - Delivers honest scarcity: "Abhi is area mein is trade mein kaam bahut kam hai"
 - Offers alternatives: slightly farther radius, adjacent trade, alert signup
 - Captures the gap as a signal back to ONEST — this is KKB's contribution to the network

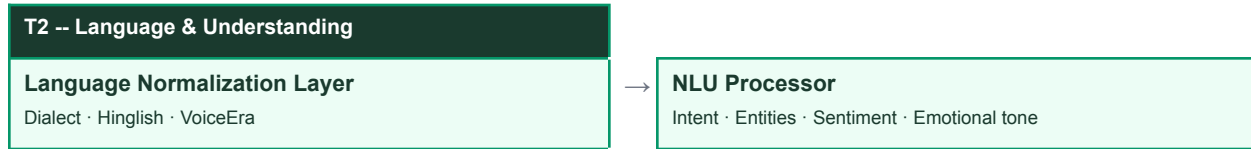
12. KKB System Wiring — DPG Block Diagram

All 15 blocks and their connections. Arrows show data flow direction. Dotted lines show observability sidecars.

Channels (Entry Points)

 VOIP (5226) Inbound + Outbound	 WhatsApp Handoff + Re-engagement	 Outbound Dialer Campaign Orchestrator driven
--	--	--

↓ all channels normalised at entry



↓ structured NLU payload passed to safety layer

T3 -- Safety & Trust		
Guardrails Engine (Input) Topic check · PII · Abuse filter	HITL Handler Loop ≥ 3 · Distress · User request	Guardrails Engine (Output) KKB brand DNA · No guarantees · Source check

↓ clean input forwarded to memory & orchestration

T1 -- Memory & Knowledge				
Session Memory Handler This call's context	User Profile Store Full entity map across sessions	Task State Manager Mental state + journey tracking	Static Knowledge Base Schemes · Trades · Institutes	Dynamic Data Connector ONEST Blue Dot live API

↓ context assembled and passed to Manager Agent

T4 -- Intelligence & Orchestration		
Manager Agent Rules-first routing · LLM fallback	LLM Inferencing Wrapper Generates response · KKB guardrails applied	Campaign Orchestrator All re-engagement outbound flows

↓ response verified by output guardrail · actions executed · outcomes tracked

T5 -- Actions & Integration		T6 -- Observability & Learning	
Actions Executor Job application · Course enrolment · Counsellor scheduling · Consent-gated	Conversation Outcome Tracker Structured post-call outcomes to CRM	LLM Observability Full session audit log	Feedback Signal Collector Mismatch + ghost signals → ONEST quality

13. Configuration Sketch

The entire KKB deployment is defined in one YAML file. A technical implementer familiar with APIs can configure this in a day.

```
# Kaam Ki Baat — KKB deployment config
agent_name: Kaam Ki Baat
dial_number: '5226'
missed_call_callback: true

channels:
  - voip          # Primary: 5226
  - whatsapp      # Secondary: handoff and re-engagement
  - outbound_call # Re-engagement campaigns

languages:
  primary: [hindi, kannada, english]
  normalization: bhashini # Language Normalization Layer wraps Bhashini

knowledge_base:
```

- ./content/labour_schemes.pdf # PMKVY, NAPS, apprenticeship rules
- ./content/trade_descriptions.pdf # All ITI trades described simply
- ./content/training_institutes.csv # District-wise institute catalogue
- ./content/bridge_income_options.pdf # For high-urgency / skill-gap cases

live_data:

- name: onest_market_signal
 type: onest_blue_dot
 endpoint: https://api.onest.network/v2/blue-dots
 query_params:
 freshness_threshold_hours: 24
 require_verified: true
 trigger_intent: market_truth_query
 fallback: honest_scarcity_response # Never hallucinate

routing_rules:

Manager Agent config

- intent: market_truth_query → [onest_market_signal, static_kb]
- intent: scheme_query → [static_kb]
- intent: training_query → [static_kb, onest_market_signal]
- intent: apply_now → [actions_executor]
- intent: counsellor_request → [hitl_handler]
- intent: whatsapp_handoff → [channel_adapter]
- intent: unknown → [llm_router] # LLM fallback

guardrails:

blocked_phrases: [guaranteed, 'pakka milega', 'best opportunity',
 'don't worry', '100% placement']
 required_framing:
 use_ranges_not_absolutes: true # '14,000-18,000' not '18,000'
 cite_source: blue_dot # Every role must map to a Blue Dot
 time_bound_language: true # 'abhi is waqt' not 'hamesha'
 scarcity_response: honest # Never generate roles that don't exist

hitl:

trigger_on_loop_count: 3 # Evaluation loop >= 3 sessions
 trigger_on_distress: true # Emotional distress detected
 trigger_on_request: true # User asks for human
 counsellor_callback_sla_hours: 24

observability:

session_log: full
 metrics: [informed_decision_rate, days_to_first_relevant_option,
 false_positive_rate, return_call_rate, employer_callback_rate,
 counsellor_overflow_rate, onest_signal_contribution_count]
 drop_off_capture: true
 mismatch_signal_to_onest: true